



UNIVERSIDAD DE GRANADA

Facultad de Ciencias

Máster Universitario en Física: Radiaciones, Nanotecnología,
Partículas y Astrofísica

Complementos Matemáticos y Numéricos

Transformada de Fourier Discreta y Rápida

Por:

Ali-Salem Amari

Resumen

Se estudia la transformada discreta de Fourier y su implementación eficiente mediante el algoritmo de transformada rápida de Fourier (FFT) mediante implementaciones propias del algoritmo en sus versiones recursiva e iterativa. Los resultados obtenidos se comparan con soluciones analíticas y con la rutina `np.fft()` de la librería NumPy para distintas funciones representativas. Asimismo, se verifica numéricamente el cumplimiento de las propiedades fundamentales de la transformada de Fourier. Finalmente, se formula la aplicación de la FFT en el contexto de la mecánica cuántica y la resolución numérica de la ecuación de Schrödinger mediante métodos espectrales.

1 Introducción

La transformada de Fourier (DFT) directa en versión discreta se define como [1]

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}, \quad (1)$$

y la transformada inversa

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j\frac{2\pi}{N}kn},$$

con $x[n], n = 0, 1, \dots, N-1$ y $k = 0, 1, \dots, N-1$. En forma matricial

$$X = W_N x,$$

donde la matriz W tiene elementos $W_N^{k,n} = e^{-j\frac{2\pi}{N}kn}$. Es decir, calcular la transformada de Fourier de una señal tiene un coste $\mathcal{O}(N^2)$. La transformada de Fourier rápida (FFT) logra reducir este elevado coste a $\mathcal{O}(N \log N)$ [2].

1.1 Transformada de Fourier Rápida

La transformada de Fourier rápida se basa en dividir las componentes del vector x en índices pares e impares hasta llegar a la condición $N = 1$ donde $FFT(a) = a$. Por ello, esta versión rápida del algoritmo requiere que la dimensión del vector sea una potencia de 2 ($N = 2^m$).

Dividiendo la suma 1 en términos pares e impares se obtiene

$$\begin{aligned} X[k] &= \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn} = \sum x[n] e^{-j2\pi kn/(N)} + \sum x[2n+1] e^{-j2\pi k(2n+1)/(N)} \\ &= \sum x_{par}[n] e^{-j2\pi kn/(N/2)} + e^{-j2\pi k/N} \sum x_{impar}[n] e^{-j2\pi kn/(N/2)}. \end{aligned}$$

Definiendo

$$P[k] = DFT[x_{par}], \quad I[k] = DFT[x_{impar}],$$

se obtiene

$$X[k] = P[k] + W_N^k I[k]$$

donde $W_N^k = e^{-j2\pi k/N}$. Solo es necesario hacer los cálculos para la mitad del espectro, ya que $e^{-j2\pi(k+N/2)n/N} = e^{-j2\pi kn/N} e^{-j2\pi(N/2)n/N}$, y $e^{-j2\pi(N/2)n/N} = e^{-j\pi n} = (-1)^n$. Por lo que

$$X[k + N/2] = P[k] - W_N^k I[k].$$

Si realizamos este mismo proceso para las transformadas de Fourier $P[k]$ y $I[k]$ de tamaño $N/2$, y de la misma manera para las nuevas transformadas que surgen se llega a $N = 1$, donde la DFT es trivial.

En cada nivel hay N operaciones, y el número de niveles es $\log_2 N$, por lo que el coste total es $\mathcal{O}(N \log N)$.

Ahora se proponen dos métodos distintos para implementar la DFT.

1.1.1 Método recursivo

El método recursivo hace que la función se llame a sí misma varias veces (haciendo la combinación de par e impar) hasta llegar al caso base (el trivial). Aunque presenta la ventaja de ser muy fácil de implementar, requiere crear nuevos vectores a cada subdivisión, lo que implica usar más memoria de computadora. Usando la librería NumPy (*ver.* 1.26.4) [3] en Python (*ver.* 3.12.10) la función es

```
import numpy as np

def fft_rec(x, inverse=False):

    x = np.asarray(x, dtype=complex)
    N = x.shape[0]

    if N == 1:
        return x.copy()

    if N % 2 != 0:
        raise ValueError("N must be a power of 2")

    sign = 1 if inverse else -1

    # Recursion
    X_even = fft_rec(x[::2], inverse)
    X_odd = fft_rec(x[1::2], inverse)

    X = np.zeros(N, dtype=complex)

    # Twiddle factors ( $W_{k+1} = W_k * W_1$ )
    W1 = np.exp(sign * 2j * np.pi / N)
    W = 1.0 + 0j

    for k in range(N // 2):

        t = W * X_odd[k]

        X[k] = X_even[k] + t
        X[k + N // 2] = X_even[k] - t

        W *= W1

    if inverse:
        X /= 2

    return X
```

Listing 1: Función que implementa la FFT con el método recursivo.

1.1.2 Método iterativo

El método iterativo trabaja sobre el mismo vector x , por lo que no adolece del problema de la versión recursiva. La FFT implica ir seleccionando elementos pares e impares sucesivamente, esto se puede implementar fácilmente usando el *bit-reversal*.

En el primer nivel, la FFT selecciona las componentes pares e impares, lo que en binario significa tomar todas las componentes que acaben en 0 para los pares y en 1 para los impares. En el siguiente nivel se elimina el último bit y se vuelven a seleccionar pares e impares por penúltimo bit, y así sucesivamente hasta llegar al último nivel. Por ejemplo, para un vector de dimensión $N = 8$ el algoritmo es

- Nivel 0: $(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8)$
- Nivel 1: $(x_0, x_2, x_4, x_6 | x_1, x_3, x_5, x_7)$
- Nivel 2: $(x_0, x_4 | x_2, x_6 | x_1, x_5 | x_3, x_7)$
- Nivel 3: $(x_0 | x_4 | x_2 | x_6 | x_1 | x_5 | x_3 | x_7)$

El resultado final es que la componente en el bit $b_{N-1} \cdots b_1 b_0$ acaba en la componente $b_0 b_1 \cdots b_{N-1}$, es decir, invierte su bit posicional. Una vez obtenido el vector invertido el algoritmo reconstruye la transformada combinando progresivamente los valores mediante las exponenciales

$$W_8^k = e^{-2\pi i k / 8}.$$

La reconstrucción se realiza en tres etapas ($2^3 = 8$).

En la etapa 1 se agrupan los datos de dos en dos y se calculan transformadas de tamaño 2

$$\begin{aligned} a_0 &= x_0 + x_4, & b_0 &= x_0 - x_4, \\ a_1 &= x_2 + x_6, & b_1 &= x_2 - x_6, \\ a_2 &= x_1 + x_5, & b_2 &= x_1 - x_5, \\ a_3 &= x_3 + x_7, & b_3 &= x_3 - x_7. \end{aligned}$$

Tras esta etapa se obtiene

$$(a_0, b_0, a_1, b_1, a_2, b_2, a_3, b_3).$$

En la etapa 2 se combinan ahora bloques de cuatro elementos utilizando los factores

$$W_4^k = e^{-2\pi i k / 4}, \quad k = 0, 1.$$

Para el primer bloque

$$\begin{aligned} c_0 &= a_0 + W_4^0 a_1 = a_0 + a_1, \\ c_1 &= b_0 + W_4^1 b_1 = b_0 - ib_1, \\ c_2 &= a_0 - W_4^0 a_1 = a_0 - a_1, \\ c_3 &= b_0 - W_4^1 b_1 = b_0 + ib_1, \end{aligned}$$

y de forma análoga para el segundo bloque

$$\begin{aligned}c_4 &= a_2 + a_3, \\c_5 &= b_2 - ib_3, \\c_6 &= a_2 - a_3, \\c_7 &= b_2 + ib_3.\end{aligned}$$

Tras esta etapa se obtiene

$$(c_0, c_1, c_2, c_3, c_4, c_5, c_6, c_7).$$

Finalmente, se combinan todos los coeficientes utilizando los factores

$$W_8^k = e^{-2\pi i k/8}, \quad k = 0, 1, 2, 3.$$

Se obtiene

$$\begin{aligned}X_0 &= c_0 + W_8^0 c_4, & X_4 &= c_0 - W_8^0 c_4, \\X_1 &= c_1 + W_8^1 c_5, & X_5 &= c_1 - W_8^1 c_5, \\X_2 &= c_2 + W_8^2 c_6, & X_6 &= c_2 - W_8^2 c_6, \\X_3 &= c_3 + W_8^3 c_7, & X_7 &= c_3 - W_8^3 c_7.\end{aligned}$$

El vector final

$$(X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7)$$

coincide con la transformada discreta de Fourier del vector original.

Cada etapa duplica el tamaño de las subtransformadas previamente calculadas, introduciendo las fases W_N^k . Tras $\log_2 N$ etapas, se obtiene la DFT completa con coste $\mathcal{O}(N \log N)$. El siguiente código implementa la FFT iterativa para un tamaño N genérico

```
import numpy as np

def fft_it(x, inverse=False):

    x = np.asarray(x, dtype=complex).copy()
    N = x.shape[0]

    if (N & (N - 1)) != 0:
        raise ValueError("N must be a power of 2")

    # -----
    # 1) BIT REVERSAL
    # -----

    nbits = int(np.log2(N))

    def bit_reverse(n, bits):
        r = 0
        for _ in range(bits):
            r = (r << 1) | (n & 1)
```

```

        n >= 1
        return r

    for i in range(N):
        j = bit_reverse(i, nbits)
        if j > i:
            x[i], x[j] = x[j], x[i]

    # -----
    # 2) STAGES
    # -----

    sign = 1 if inverse else -1

    m = 2
    while m <= N:

        half = m // 2
        Wm = np.exp(sign * 2j * np.pi / m)

        for k in range(0, N, m):

            W = 1.0 + 0j

            for j in range(half):

                t = W * x[k + j + half]
                u = x[k + j]

                x[k + j] = u + t
                x[k + j + half] = u - t

            W *= Wm

        m *= 2

    # -----
    # 3) IFFT NORMALISATION
    # -----

    if inverse:
        x /= N

    return x

```

Listing 2: Función que implementa la FFT con el método iterativo.

2 Casos particulares

En esta sección se estudian transformadas de Fourier para varias funciones típicas usando el código presentado y se comparan con la rutina `np.fft()` de la librería NumPy. El error se mide como la norma L^2 de la diferencia entre la solución analítica y la numérica.

En todos los casos se observa una excelente correspondencia con la solución analítica así como un error del orden de 10^{-10} para los casos iterativo y recursivo.

2.1 Seno y coseno

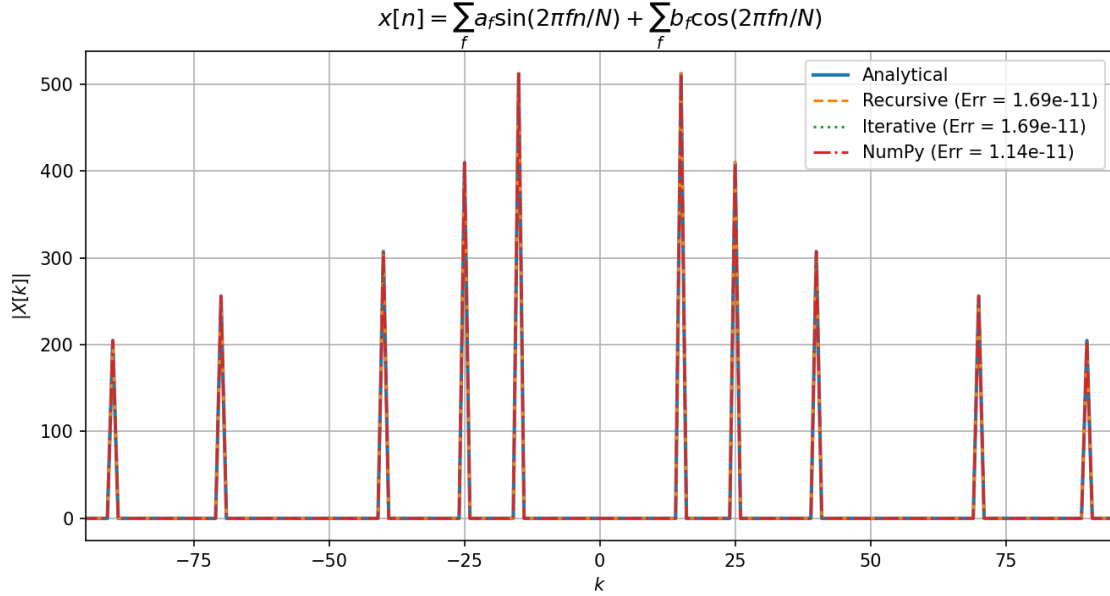


Figura 1: Comparación de la DFT implementada con los códigos descritos en la sección 1, la dada por la rutina `np.fft()` y la solución analítica de una función combinación de senos y cosenos. Se han tomado $N = 1024$ puntos para discretizar la función. El error se mide como la norma L^2 .

2.2 Gaussianas

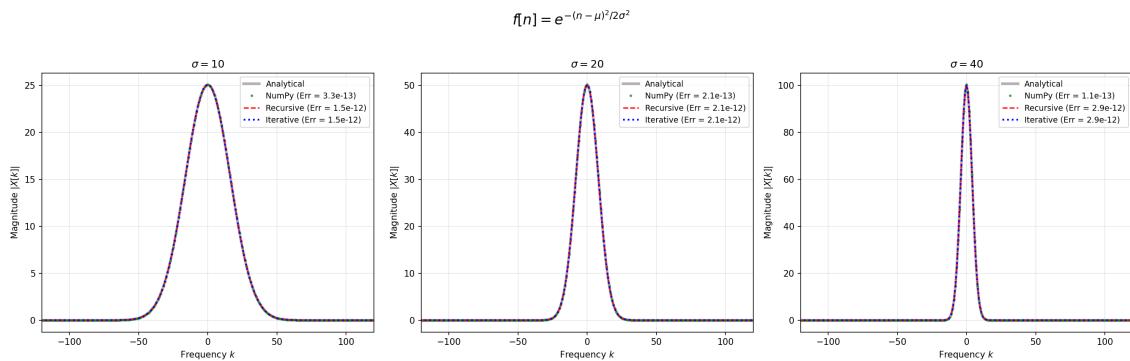


Figura 2: Comparación de la DFT implementada con los códigos descritos en la sección 1, la dada por la rutina `np.fft()` y la solución analítica de funciones gaussianas. Se han tomado $N = 1024$ puntos para discretizar la función. El error se mide como la norma L^2 .

2.3 Deltas de Kronecker

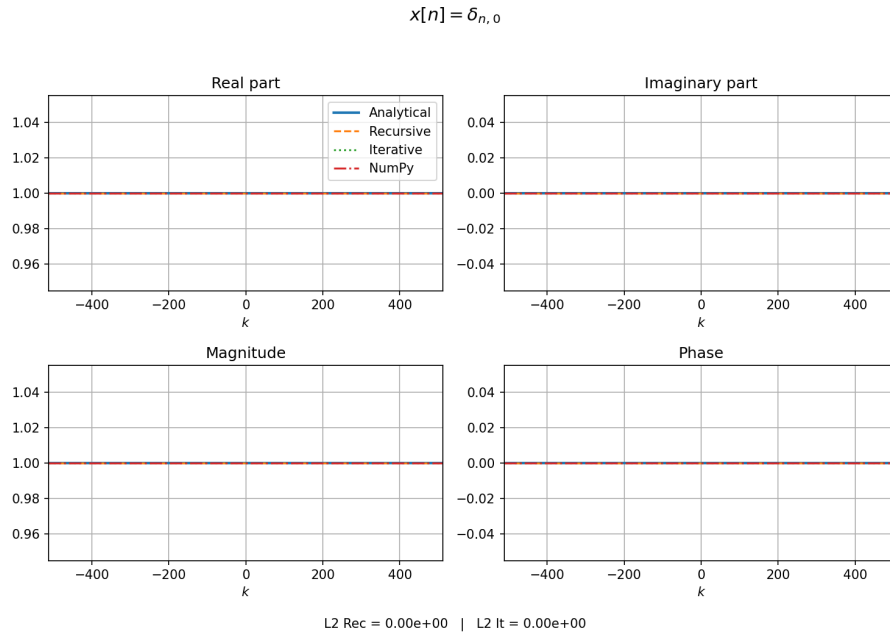
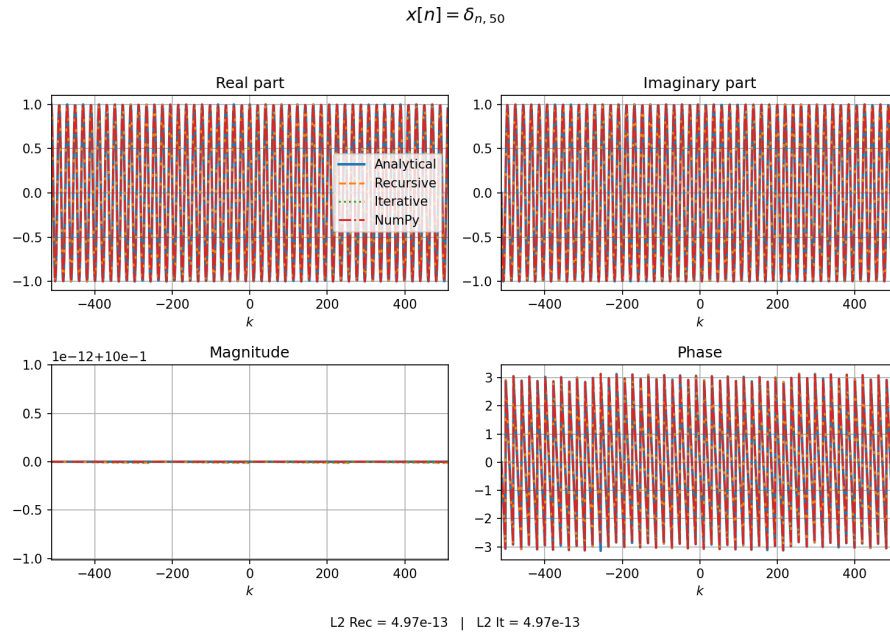
(a) $\delta_{n,0}$ (b) $\delta_{n,50}$

Figura 3: Comparación de la DFT implementada con los códigos descritos en la sección 1, la dada por la rutina `np.fft()` y la solución analítica de funciones delta de Kronecker. Se han tomado $N = 1024$ puntos para discretizar la función. El error se mide como la norma L^2 .

2.4 Pulso cuadrado

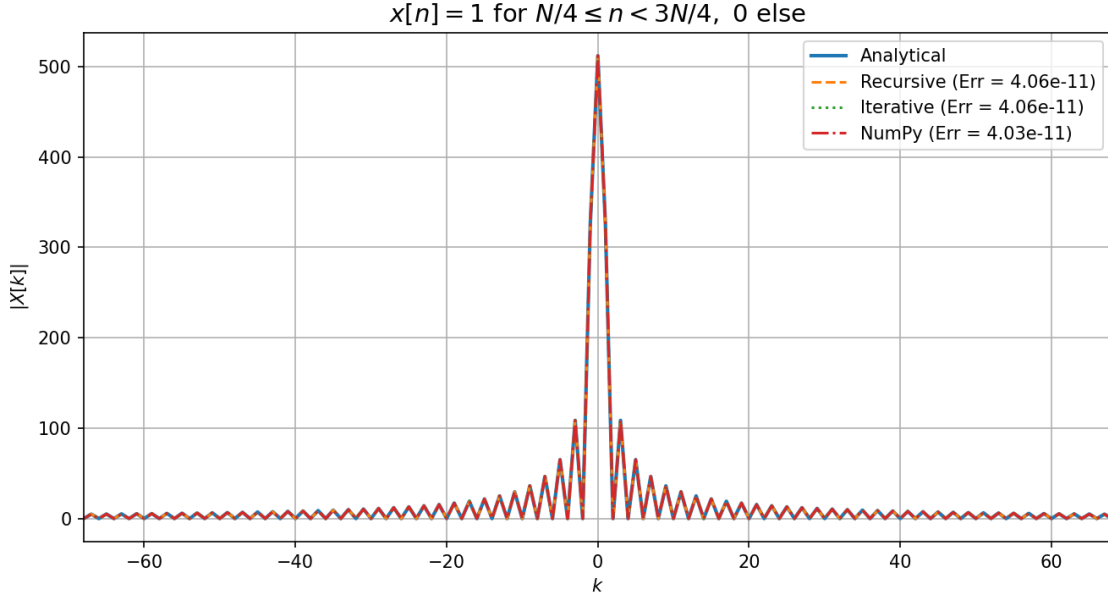


Figura 4: Comparación de la DFT implementada con los códigos descritos en la sección 1, la dada por la rutina `np.fft()` y la solución analítica de un pulso cuadrado. Se han tomado $N = 1024$ puntos para discretizar la función. El error se mide como la norma L^2 .

2.5 Función constante

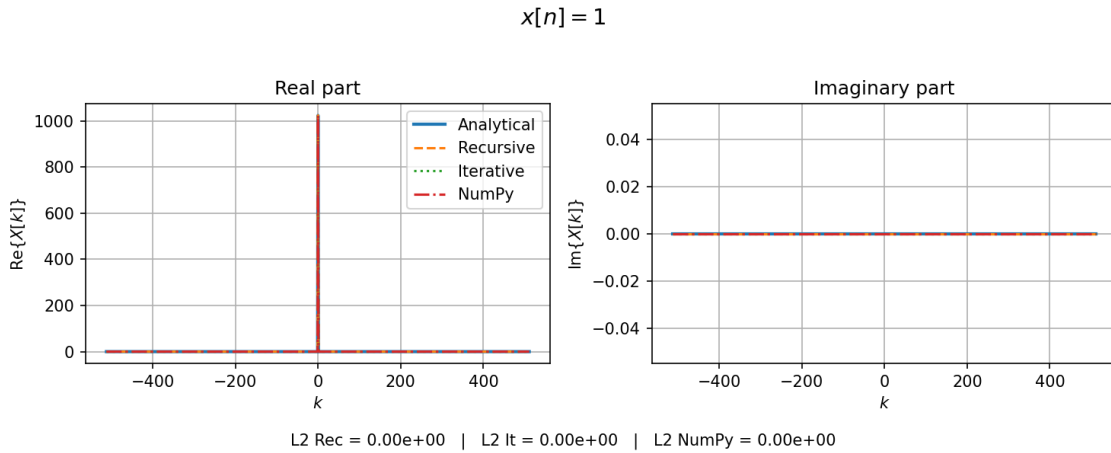


Figura 5: Comparación de la DFT implementada con los códigos descritos en la sección 1, la dada por la rutina `np.fft()` y la solución analítica de una constante. Se han tomado $N = 1024$ puntos para discretizar la función. El error se mide como la norma L^2 .

2.6 Onda plana

$$x[n] = e^{j2\pi k_0 n/N}$$

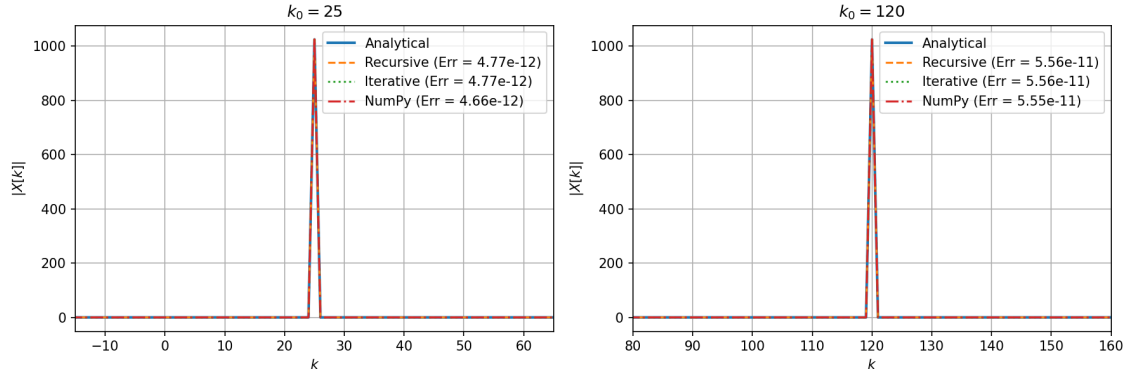


Figura 6: Comparación de la DFT implementada con los códigos descritos en la sección 1, la dada por la rutina `np.fft()` y la solución analítica de una onda plana compleja $x[n] = e^{j2\pi k_0 n/N}$. Se han tomado $N = 1024$ puntos para discretizar la función. El error se mide como la norma L^2 .

3 Verificación de propiedades de la FFT

La transformada de Fourier teórica verifica las siguientes propiedades

- $\mathcal{F}\mathcal{F}^{-1} = \mathcal{F}^{-1}\mathcal{F} = \mathcal{I}$,
- $\mathcal{F}^4 = N^2 \mathcal{I}$,
- Conservación de la norma (Parseval).

En la siguiente gráfica se muestra el error medido en norma L^2 de cada una de las propiedades descritas. Se observa que pese a aumentar con N el error se mantiene del orden de 10^{-11} , por lo que las propiedades teóricas se verifican.

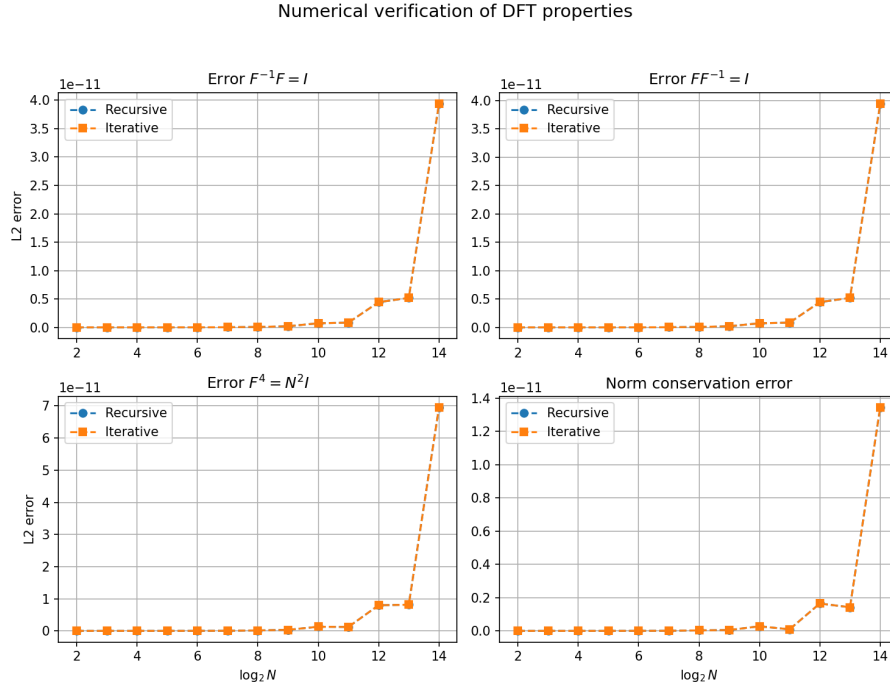


Figura 7: Comprobación de las propiedades teóricas de la DFT implementada con los códigos descritos en la sección 1. Se ha tomado como señal un vector complejo aleatorio con componentes distribuidas según una normal de media 0 y varianza 1. Los resultados se promediaron sobre 10^2 realizaciones. El eje x está en escala $\log_2 N$. El error se mide como la norma L^2 .

La última comprobación es el tiempo de ejecución, que tiene una complejidad de orden $\mathcal{O}(N \log_2 N)$. Como se puede observar en la siguiente gráfica, al normalizar el tiempo por $N \log_2 N$ se obtienen curvas constantes, lo que verifica que efectivamente el tiempo escala con $N \log_2 N$. Además, no se aprecia una diferencia significativa en tiempo de ejecución entre el método recursivo e iterativo.

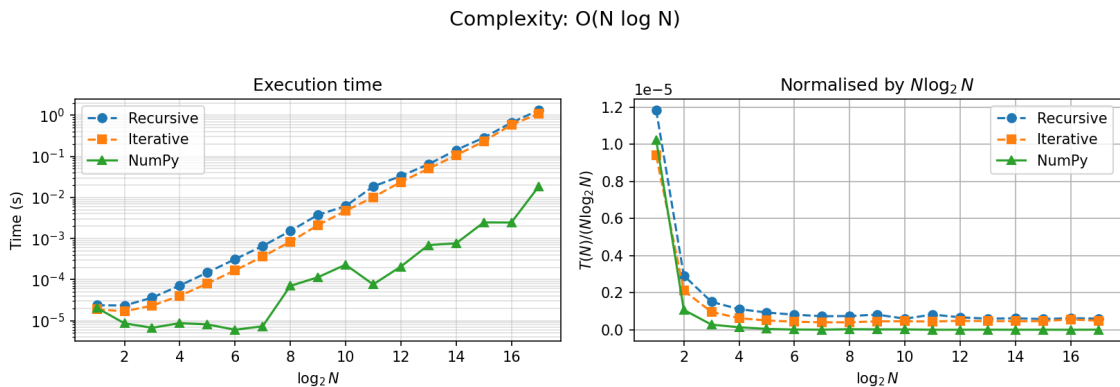


Figura 8: Comprobación del grado de complejidad la DFT implementada con los códigos descritos en la sección 1 y `np.fft()`. Se ha tomado como señal un vector complejo aleatorio con componentes distribuidas según una normal de media 0 y varianza 1.

4 Transformada de Fourier en Física Cuántica

La función de onda ψ de un estado cuántico puede representarse tanto en el espacio de posiciones x como en el espacio de momentos p . Ambas representaciones están relacionadas a través de la transformada de Fourier [5]

$$\phi(p) = \frac{1}{\sqrt{2\pi\hbar}} \int_{-\infty}^{+\infty} \psi(x) e^{-jpx/\hbar} dx,$$

y su inversa

$$\psi(x) = \frac{1}{\sqrt{2\pi\hbar}} \int_{-\infty}^{+\infty} \phi(p) e^{jpx/\hbar} dp.$$

4.1 Caso estacionario

La ecuación de Schrödinger estacionaria para una partícula en una dimensión viene dada por

$$\hat{H}\psi(x) = E\psi(x), \quad (4.1)$$

donde el Hamiltoniano es

$$\hat{H} = -\frac{\hbar^2}{2m} \frac{d^2}{dx^2} + V(x). \quad (4.2)$$

El operador cinético contiene una derivada segunda, que resulta costosa de evaluar directamente en el espacio real al involucrar puntos x_{n+1} , x_n y x_{n-1} . Sin embargo, en el espacio de Fourier es muy sencillo de evaluar

$$\frac{d^2}{dx^2} \longleftrightarrow -k^2, \quad (4.3)$$

por lo que el operador cinético se vuelve diagonal en dicha representación. Se puede así llevar $\psi(x)$ al espacio de momentos mediante una transformada de Fourier, evaluar el operador cinético en dicho espacio (donde es diagonal) y volver al espacio real con una transformada inversa. Es decir

$$\hat{T} = \mathcal{F}^{-1} \left(\frac{\hbar^2 k^2}{2m} \right) \mathcal{F}. \quad (4.4)$$

Por otro lado, el potencial actúa en el espacio de posiciones como

$$(\hat{V}\psi)(x) = V(x)\psi(x), \quad (4.5)$$

es decir, ya es diagonal. El Hamiltoniano discreto completo queda entonces

$$H = F^{-1}DF + V. \quad (4.6)$$

4.2 Caso dependiente del tiempo

En problemas de evolución la función de onda a tiempo t tiene solución formal

$$\psi(t) = e^{-j\hat{H}t/\hbar}\psi_0.$$

El problema es que los operadores $\hat{T}$ y $\hat{V}$ no suelen conmutar. En este caso se utiliza la aproximación de tiempo corto

$$\psi(t + \Delta t) = e^{-j\hat{H}\Delta t/\hbar}\psi(t),$$

donde el operador de evolución temporal se evalúa usando la aproximación de Trotter [4]

$$e^{-j\hat{H}\Delta t/\hbar} = e^{-j\hat{T}\Delta t/2\hbar}e^{-j\hat{V}\Delta t/\hbar}e^{-j\hat{T}\Delta t/2\hbar} + \mathcal{O}(\Delta t^3).$$

De esta forma el esquema de resolución queda como

1. $\psi(x) \rightarrow \phi(p)$ con una FFT
2. Multiplicar por fase cinética
3. $\phi'(p) \rightarrow \psi'(x)$ con una IFFT
4. Multiplicar por fase potencial
5. $\psi''(x) \rightarrow \phi''(p)$ con una FFT
6. Multiplicar por fase cinética
7. $\phi'''(p) \rightarrow \psi'''(x)$ con una IFFT

4.3 Ejemplo: estado fundamental del oscilador armónico

Como aplicación concreta se calcula la energía del estado fundamental del oscilador armónico cuántico unidimensional. En unidades naturales ($\hbar = m = \omega = 1$) el Hamiltoniano es

$$\hat{H} = -\frac{1}{2}\frac{d^2}{dx^2} + \frac{1}{2}x^2,$$

con autovalor fundamental exacto $E_0 = 1/2$ y autofunción $\psi_0(x) = \pi^{-1/4}e^{-x^2/2}$.

Se emplea el método *split-operator* en tiempo imaginario: sustituyendo $t \rightarrow -i\tau$ en el operador de evolución, la exponencial $e^{-\hat{H}\tau}$ filtra las componentes de mayor energía, de modo que toda función de onda inicial con solapamiento no nulo con el estado fundamental converge a él. El operador cinético se aplica mediante FFT como se describió arriba.

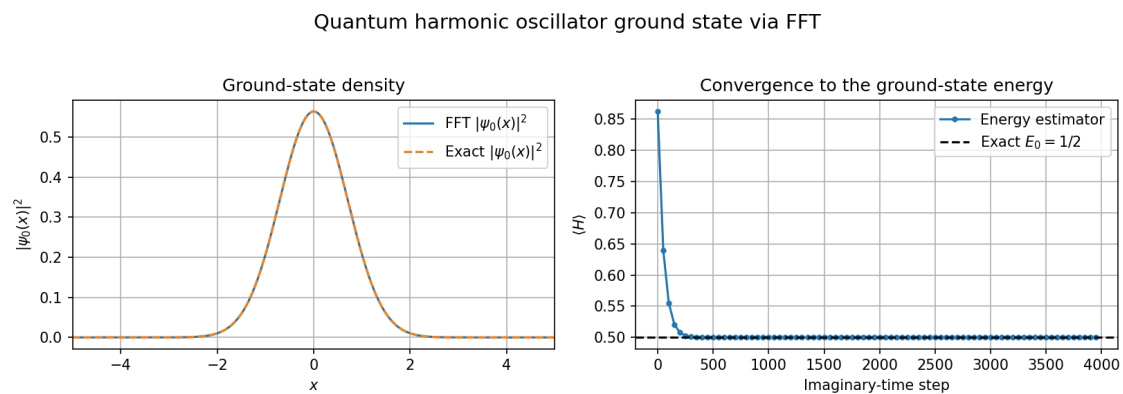


Figura 9: Estado fundamental del oscilador armónico cuántico obtenido mediante propagación en tiempo imaginario con el método *split-operator* basado en FFT. Izquierda: densidad de probabilidad $|\psi_0(x)|^2$ numérica frente a la gaussiana analítica $\pi^{-1/2}e^{-x^2}$. Derecha: convergencia del estimador de energía $\langle H \rangle$ al valor exacto $E_0 = 1/2$.

5 Conclusión

Se ha estudiado la transformada discreta de Fourier y su implementación eficiente mediante el algoritmo de transformada rápida de Fourier (FFT) que permite reducir el coste original $\mathcal{O}(N^2)$ a un orden $\mathcal{O}(N \log_2 N)$ mediante una descomposición recursiva basada en la separación de índices pares e impares.

Desarrollando dos implementaciones propias del algoritmo FFT, una versión recursiva y otra iterativa basada en el procedimiento de *bit-reversal*, se observa que ambas coinciden con la rutina `np.fft()` de la librería NumPy y con las soluciones analíticas para diversas funciones representativas. En todos los casos, los resultados obtenidos presentan una excelente concordancia, con errores del orden de 10^{-10} .

Asimismo, se ha comprobado numéricamente que los algoritmos implementados verifican las propiedades fundamentales de la transformada de Fourier, como la inversión, la conservación de la norma y la periodicidad de orden cuatro.

El estudio del tiempo de ejecución muestra que ambas versiones del algoritmo presentan un comportamiento compatible con la complejidad teórica $\mathcal{O}(N \log_2 N)$. Aunque la versión iterativa presenta ventajas en términos de uso de memoria, no se observan diferencias significativas en tiempo de ejecución para los tamaños considerados.

Finalmente, se ha explorado la aplicación de la transformada de Fourier en el contexto de la mecánica cuántica, analizando la relación entre las representaciones en posición y momento, así como la resolución numérica de la ecuación de Schrödinger mediante métodos espectrales. El uso de la FFT permite evaluar de forma eficiente el operador cinético y construir el Hamiltoniano discreto, obteniendo niveles de energía y autofunciones con alta precisión, como se ha mostrado para el oscilador armónico cuántico.

Referencias

- [1] JUAN CARLOS ANGULO IBAÑEZ. *Variable compleja: resolución de problemas y aplicaciones*. Ediciones Paraninfo, SA, 2012.
- [2] E Oran Brigham. *The fast Fourier transform and its applications*. Prentice-Hall, Inc., 1988.
- [3] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.
- [4] Malvin H Kalos and Paula A Whitlock. *Monte carlo methods*. John Wiley & Sons, 2008.
- [5] Jun John Sakurai and Jim Napolitano. *Modern quantum mechanics*. Cambridge university press, 2020.